

RAM MEMORY ARCHITECTURE DESIGN

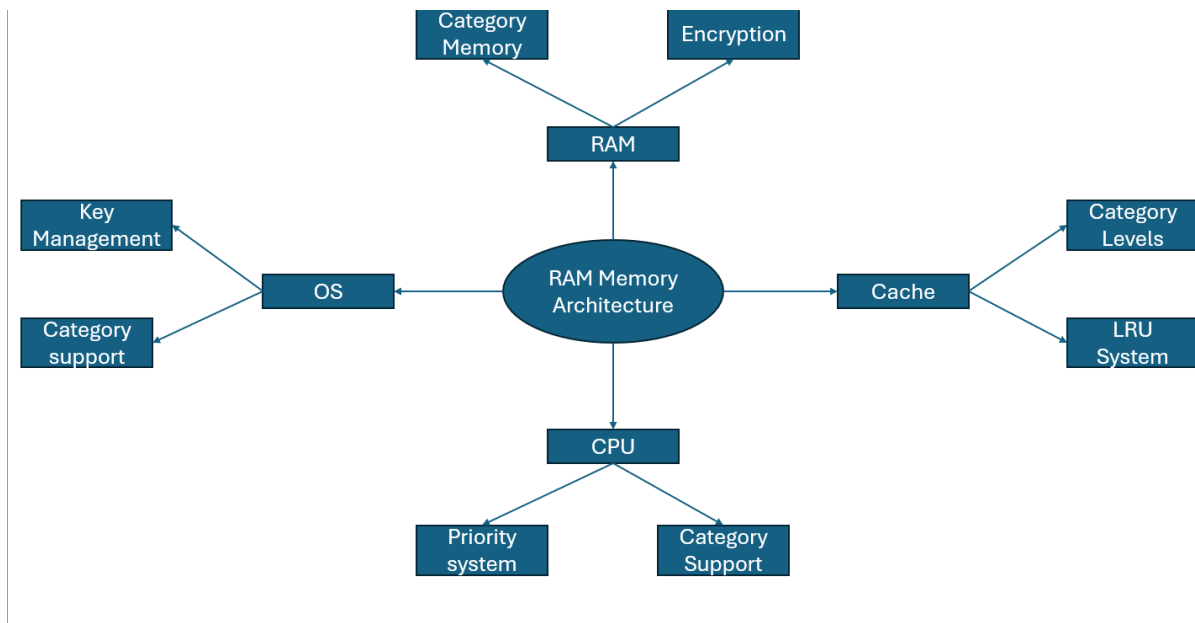
Contents

Introduction	2
RAM Category-Based Memory	3
Category-Based Memory Workflow	4
RAM Encryption Mode	5
RAM Memory Architecture Trade-offs	6

Introduction

This RAM Memory Architecture is designed to improve the performance and security of RAM. This was also designed to address the limitation of flat addressing, which RAM currently uses to store instructions. This RAM memory architecture was created because modern systems and functions such as gaming, AI/ML infrastructure, and server workloads require fast and consistent performance. This RAM Memory Architecture was created with two main features. These features are category-based memory and RAM encryption mode. This design will explain the changes needed and how each feature works.

This Memory Architecture cannot promise perfect performance or security of RAM. However, it can promise to increase performance and reduce workloads compared to the current flat addressing systems. It can also provide protection against cold boot attacks without having to write contents to encrypted disk. This is a design which may be time-consuming for engineers to implement. Some systems may only use the category-based memory or RAM encryption mode while some systems may use both. Make sure to adapt this design based on operating system and CPU architecture if necessary for your own systems.



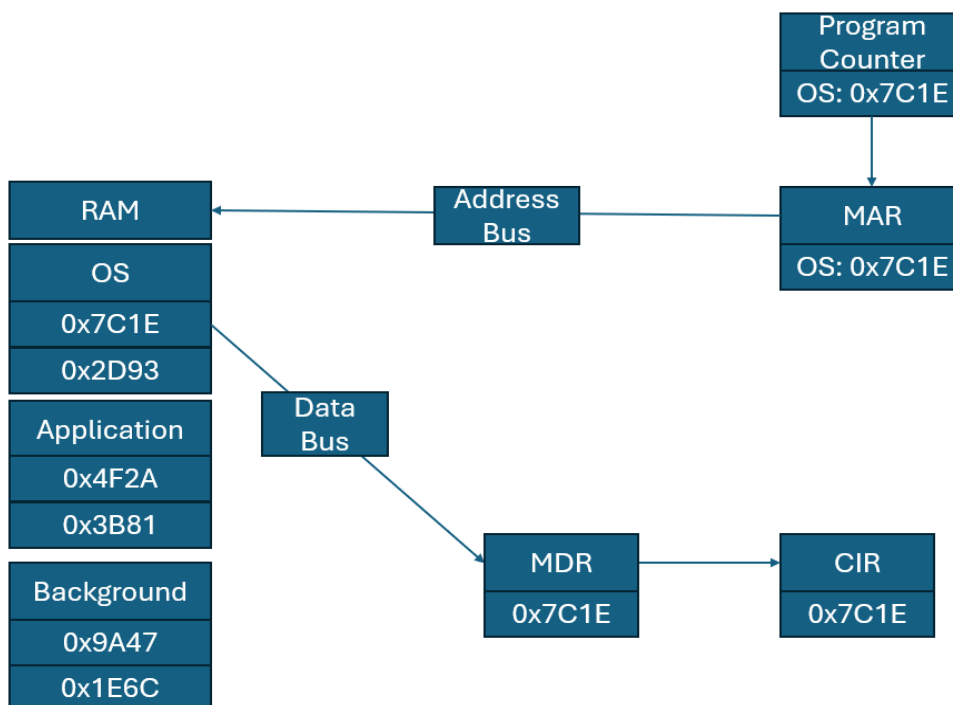
This spider diagram shows the key changes made to RAM, the CPU, Cache and operating system. For those that may not be familiar with computing architecture, LRU stands for least recently used which is used to remove less frequently used data from cache. More detail on changes for cache can be found in the cache design document in this repo.

RAM Category-Based Memory

- **Category-based memory is where instructions for RAM are stored in specific categories instead of just one space.**
- **For example, one category could be applications where instructions used by applications on a device are found.**
- **The operating system's memory controller would need to be reconfigured to support RAM's category-based memory.**
- **There are 3 main categories that should be used by default on most modern systems.**
- **These categories are operating system, applications, and background.**
- **The categories are efficient because the CPU can look for specific instructions within categories leading to faster access times.**
- **This improves performance overall as instructions are fetched faster.**
- **The CPU should also use a priority system that allows the CPU to focus on fetching certain instructions.**
- **This priority system can be created by using a scheduling algorithm that may focus on operating system or applications more.**
- **This priority system may also be used for assigning RAM space for different categories.**
- **For example, in an 8GB RAM module, OS is given 4 GB, App is given 3 GB and Background is given 1 GB**
- **CPU components like the Program Counter, Address Bus, Memory Address Register and Control Unit will need to be configured to support category-based memory.**
- **The program counter would need to be configured to show the category and the address of the next instruction in memory.**
- **The address bus would need to be configured to search for specific categories and then navigate the categories for the specific address.**
- **The control unit may need to be configured to manage the dataflow between categories.**
- **The memory address register must also be configured to store category and address.**
- **Category-based memory can be supported on mobile and PC devices.**
- **For mobile devices, overheating becomes more of a risk.**
- **This is because the CPU is fetching instructions faster, which can mean more instructions being executed per cycle producing heat.**
- **Implementation for category-based memory would take years and require coordination between hardware and OS vendors.**

Category-Based Memory Workflow

- This workflow is designed to show engineers or users how the CPU process would work.
- First, the program counter points to the next category and address in memory.
- For example, this could look like OS: 0x7C1E within the program counter.
- Second, the category and address are then temporarily stored in the MAR.
- MAR stands for memory address register for those who may not know.
- Third, the address bus heads to the OS category in RAM.
- The address bus then navigates the OS category to find the specific address.
- Fourth, once the address is found, the data bus transports the instruction.
- This instruction is stored temporarily in the memory data register, also known as MDR.
- Fifth, the instruction is then sent to the current instruction register also known as CIR.
- In the CIR, the instruction is decoded into operand and opcode.
- Opcode tells the CPU what to do with data, while operand tells the CPU what data to perform that action on.
- Sixth, the CPU then executes the instruction using other components such as the ALU where needed.
- Finally, the program counter increments by one and points to the next category and address in memory.
- For example, this could look like App: 0x4F2A



The spider diagram above shows what RAM Architecture looks like when using category-based memory. This diagram does not include cache due to the focus on RAM.

RAM Encryption Mode

- **RAM encryption mode is where the contents of RAM are encrypted while the device is in a certain state.**
- **This RAM encryption mode works differently from traditional hibernation mode.**
- **Traditional hibernation mode writes the contents of RAM to the encrypted disk which is slower and is dependent on disk encryption and authentication.**
- **RAM encryption mode encrypts RAM in place during hibernation mode or a certain state.**
- **The RAM contents are decrypted instantly when the user reauthenticates.**
- **The encryption and decryption process is managed by the operating system.**
- **This is because operating systems already manage the key for device encryption.**
- **The encryption key should be stored in TPM (Trusted Platform Module).**
- **This is important because the encryption key needs to be stored separately from RAM itself.**
- **The RAM contents should be encrypted until either the user authenticates or the RAM contents are completely wiped after the device loses power.**
- **The operating system should also allow the user to decide what authentication methods or encryption algorithms they can use for encrypting the contents of RAM.**
- **For recovery, a one-time key is shown to the user which is stored securely by the OS on the device via TPM or an encrypted partition.**
- **The user is responsible for how and where they store the recovery key.**
- **There should be no physical reset switch to prevent an attacker with physical access from using a physical recovery-based attack.**
- **This RAM encryption mode focuses on preventing cold boot attacks, which is a common attack vector used by nation-state actors and sometimes high-level organised cybercriminals.**
- **The RAM encryption mode is targeted towards enterprises, high-risk systems and power users.**
- **It is important to note that this RAM encryption mode may not be compatible with mobile devices due to ARM architecture.**
- **For the user workflow, the RAM encryption mode should work like below.**
- **The user configures RAM encryption mode with AES-256 and sets the authentication method to use their OS account password.**
- **The user then places their device into hibernation mode and RAM contents should be encrypted.**
- **Once the user returns, they are asked for their account password.**
- **Once they reauthenticate, RAM contents are decrypted and the user can carry on with tasks.**

RAM Memory Architecture Trade-offs

- This RAM memory architecture has trade-offs just like any other system I have built.
- There are 5 main trade-offs that I have identified.
- Engineers may face more trade-offs during implementation as this is only a design.
- The first trade-off is compatibility.
- This is because the encryption mode may not be compatible with mobile device due to the ARM architecture.
- The encryption mode also requires the device to have an TPM which not all devices support.
- This trade-off can be managed by targeting PC/laptops, using other secure storage methods on the OS or on the cloud and focusing on encryption mode for the targeted audience.
- The second trade-off is performance.
- This is because the encryption mode can cause latency due to encrypting and decrypting the contents of RAM especially if the RAM size is large.
- This trade-off can be managed by keeping the encryption mode strictly to hibernation mode and using pre-existing operating system authentication.
- The third trade-off is lockout risk.
- This is because the encryption mode may lockout the user if they forget their password.
- This trade-off can be managed through the recovery key (No data lost) or turning off the device which means all data in RAM is lost.
- The fourth trade-off is heat.
- This is because since the CPU can fetch instructions more quickly it can also execute more instructions per cycle.
- This can cause the CPU to heat up a lot more than flat addressing.
- This trade-off can be managed by cooling systems and clock speed within the CPU.
- The fifth trade-off is legacy support.
- This is because this RAM memory architecture requires major changes for the CPU, updates to the operating system and changes to RAM.
- This can take years, and most users will not be able to adapt from current hardware easily.
- This trade-off can be managed through new operating system memory controller , new generation of CPU and a new RAM module.
- This legacy support trade-off will be the most difficult and expensive to migrate.
- There may be some more migrations that can be used for these trade-offs, I have not considered.
- Make sure to adapt migrations based on resources and budget.